

Tree-Based Models

Contents

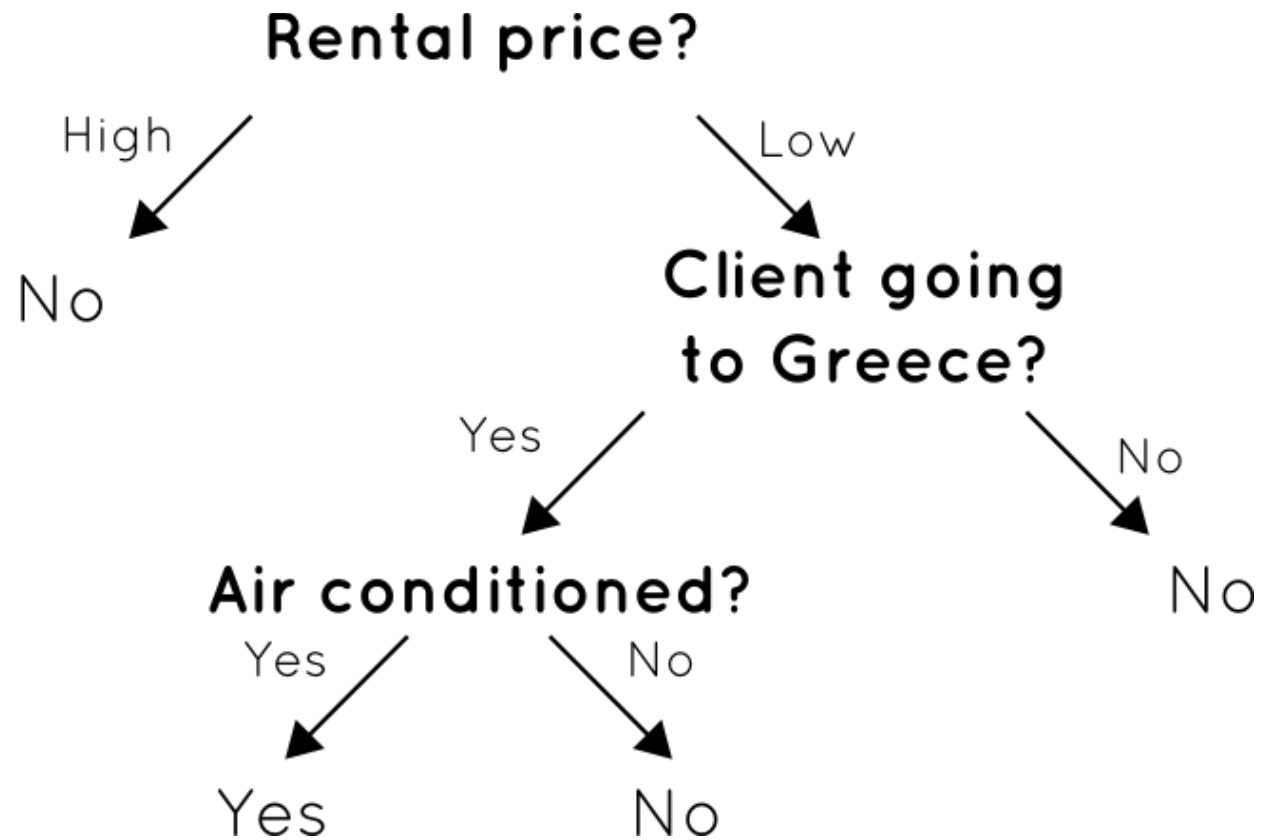
- Entscheidungsbaum (Decision Tree)
- Regularisierung durch Ensemble-Methoden
- Aufgaben

Lernziele

- Entscheidungsbäume als einfache nicht-lineare Modelle kennenlernen
- Regularisierung durch Ensemble-Methoden wie Random Forests verstehen

Entscheidungsbaum (Decision Tree)

Ist Mietfahrzeug x von Interesse für Fluggast y ?



- Ein **nicht-lineares** Verfahren für Regression oder Klassifikation
- Entscheidung kann als ein **gerichteter, geordneter Baum** dargestellt werden
- Entscheidungen folgen **hierarchisch** aufeinander

Beispiel: Palmer Penguins Datensatz

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
from os.path import join
import sys, os
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn import tree

# Import lokaler Module fürs Plotting. Bitte ignorieren
sys.path.append(os.path.abspath("src"))
from ds_helpers.plotting import setup_plots
setup_plots()
```

```
# Daten importieren
file_path = join('data', 'penguins_lter.csv')
penguins = pd.read_csv(file_path)

X_y = penguins[["Culmen Length (mm)", "Culmen Depth (mm)", "Body Mass (g)"]].dropna()

X = X_y[["Culmen Length (mm)", "Culmen Depth (mm)"]]
y = X_y["Body Mass (g)"]
```

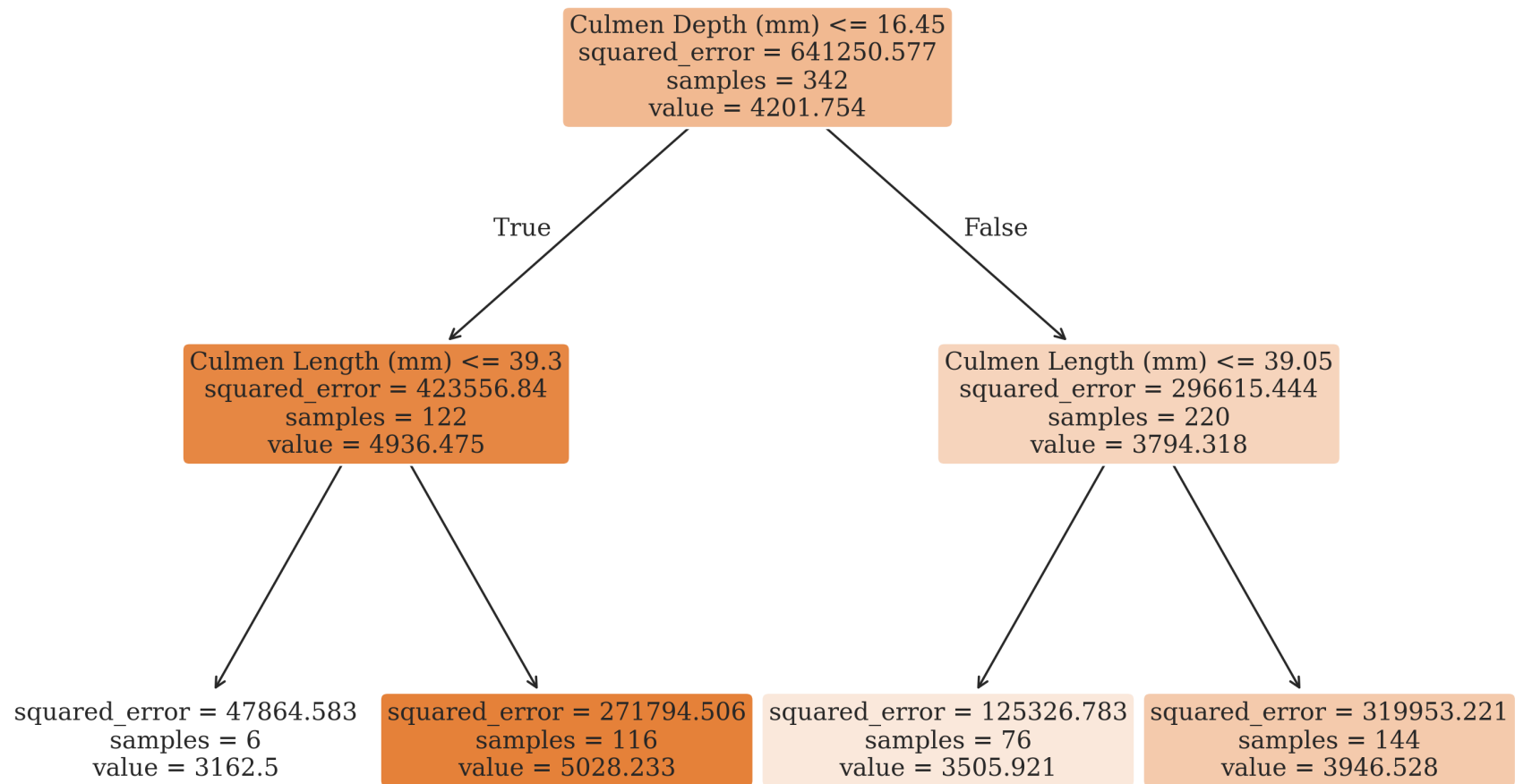
```
# Trainiere Entscheidungsbaum
dt = DecisionTreeRegressor(max_depth=2)
dt.fit(X, y)
```

▼ DecisionTreeRegressor ⓘ ?

► Parameters

<u>criterion</u>	'squared_error'
<u>splitter</u>	'best'
<u>max_depth</u>	2
<u>min_samples_split</u>	2
<u>min_samples_leaf</u>	1
<u>min_weight_fraction_leaf</u>	0.0
<u>max_features</u>	None
<u>random_state</u>	None
<u>max_leaf_nodes</u>	None
<u>min_impurity_decrease</u>	0.0
<u>ccp_alpha</u>	0.0
<u>monotonic_cst</u>	None

```
# Visualisiere Baum
fig = plt.figure(figsize=(18,12), dpi=200)
plt.tight_layout(pad=0.5)
_ = tree.plot_tree(dt, feature_names=X.columns, filled=True, rounded=True, fontsize=16)
```



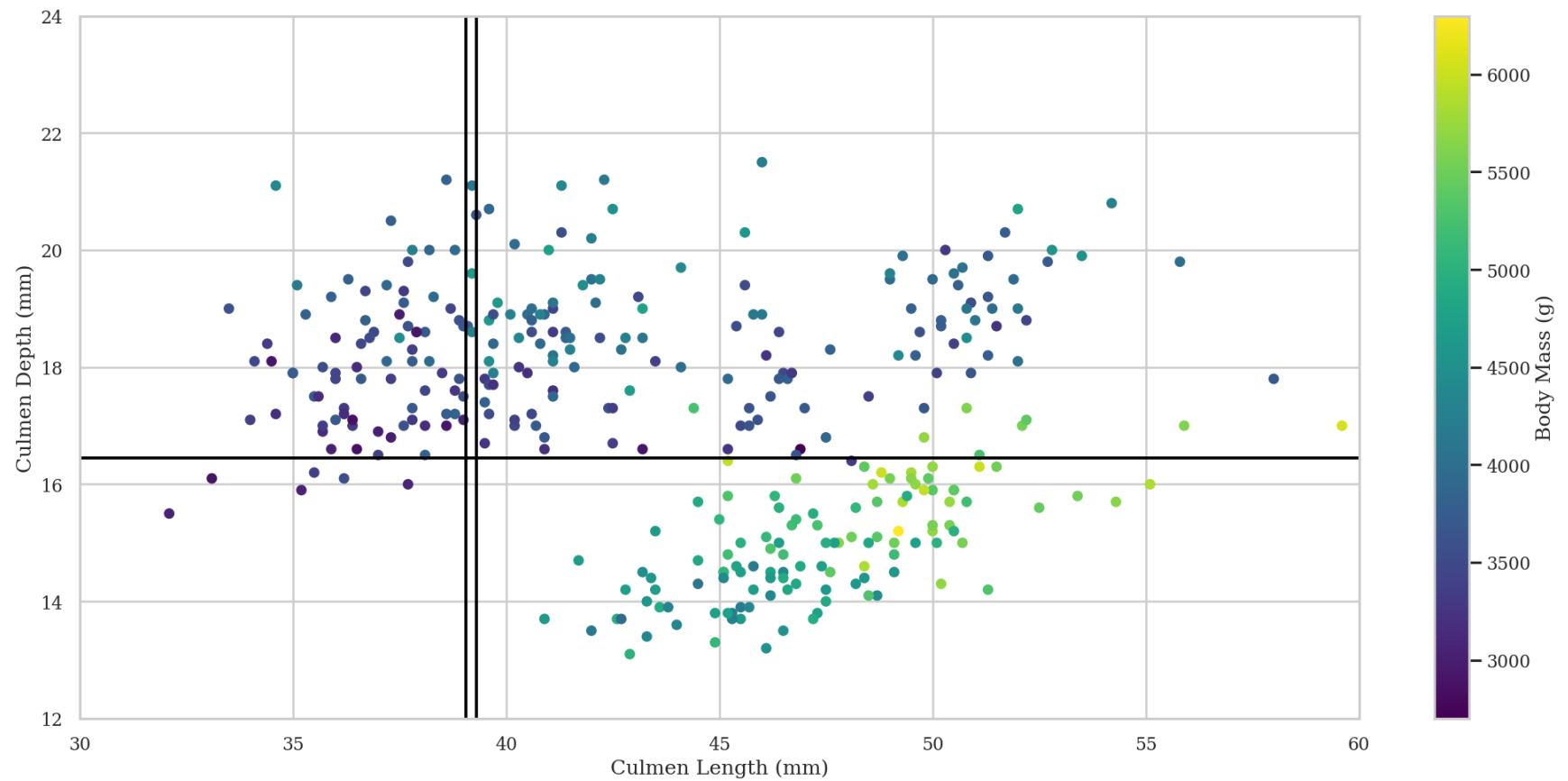
Interpretation des Baums

- 4 Pfade, 4 Blätter, 3 Knoten (den ersten Knoten bezeichnet man als *root*)
- Pfad links: Wenn Culmen Depth (mm) ≤ 16.45 und Culmen Length (mm) ≤ 39.33 , ist die Vorhersage am

niedrigsten (3162,5 g)

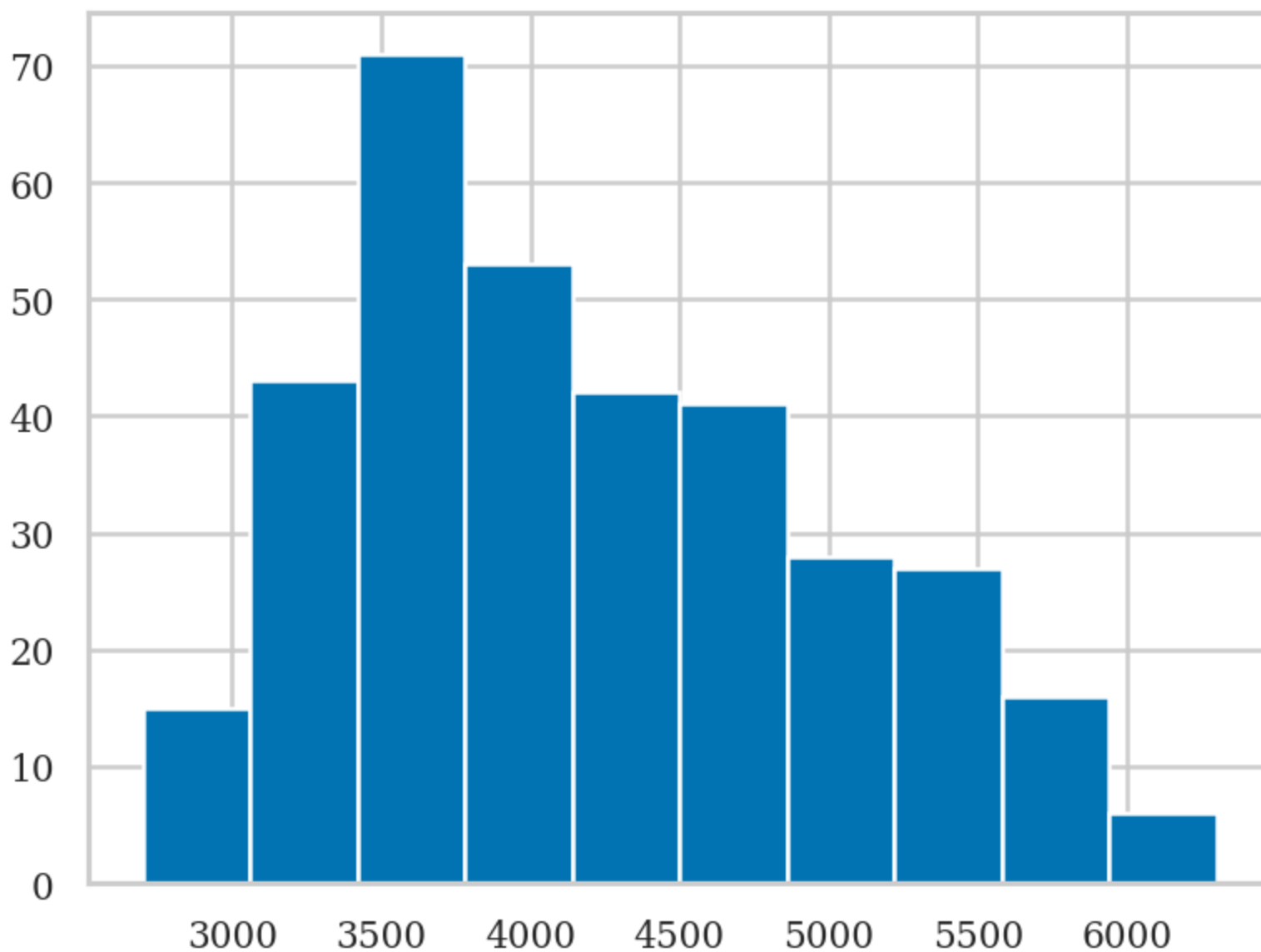
- Pfad rechts: Wenn Culmen Depth (mm) > 16.45 und Culmen Length (mm) > 39.05 , ist die Vorhersage am höchsten (3946,528 g)
- `samples`: Anzahl an Trainingsdaten in diesem Blatt
- `value`: Vorhergesagter Wert der Zielvariable für alle Daten in diesem Blatt
- `squared_error`: Der Mean Squared Error Wert pro Knoten (Metrik hängt von der gewählten Loss Function ab. Standard ist MSE für Regression)

```
# Visualisiere Aufteilung des Prädiktor-Raums
ax2 = X_y.plot.scatter(x='Culmen Length (mm)',
                      y='Culmen Depth (mm)',
                      c='Body Mass (g)', colormap='viridis',
                      s=40, figsize=(16,8), xlim=(30,60), ylim=(12,24))
ax2.axvline(39.3, color='black'); ax2.axvline(39.05, color='black')
ax2.axhline(16.45, color='black');
```



Wie gut sind die Vorhersagen des Baums?

```
X_y['Body Mass (g)'].hist();
```



```
from sklearn.metrics import mean_squared_error
mean_squared_error(X_y['Body Mass (g)'].values, (X_y['Body Mass (g)'] + 500).values)
```

250000.0

Wenn die Prognose pro Datenpunkt um 500 nach oben abweichen würde, wäre der `squared_error` (MSE) geringer als die vom trainierten Entscheidungsbaum erreichten Werte in 2 von 4 Blättern.

Algorithmus

- **Schritt 1 (Teil des Trainings):** Teile den Prädiktor-Raum in k **sich nicht-überschneidende** ' p -dimensionale Rechtecke' ein: $R_1, \dots, R_k$ (p ist die Anzahl an Prädiktoren)
- **Schritt 2 (Anwendung):** Jeder Datensatz, welcher in Region R_j landet, erhält die gleiche Prognose, und zwar den **Mittelwert** der Zielvariable von allen zu R_j zugehörigen Trainingsdaten

Beispiel

- Angenommen, es gibt zwei Regionen, R_1 und R_2 . Trainingsdaten (Pinguine) in R_1 hatten im Durchschnitt 3000g Gewicht, in R_2 waren es 3900.
- Falls ein neuer Datenpunkt in R_1 landet, prognostiziere 3000g, sonst 3900.

Training (Regressionsbaum)

Ziel: Finde Rechtecke / Boxen $R_1, \dots, R_k$, so dass der **Residual Squared Error (RSS)** minimiert wird:

$$\sum_{j=1}^k \sum_{R_j} (y_i - \hat{y}_{R_j})^2$$

- Residual Sum of Squares (RSS): nicht-normierter MSE

- $\hat{y}_{R_j}$ ist der Mittelwert der Werte der Zielvariable für alle Trainingsdaten in R_j
- Innere Summe: Summe der quadratischen Fehler für alle Trainingsdaten in Rechteck R_j
- Äußere Summe: Summe über alle Rechtecke

Problem: es ist rechnerisch nicht möglich, jede mögliche Aufsplittung des Raums in Rechtecke zu betrachten

Lösung: Recursive binary splitting: top-down greedy approach

- **top-down:** startet ganz oben an der Baum-Spitze, wo noch alle Daten in einem Rechteck liegen, und teilt den Prädiktor-Raum sukzessive auf.
- **greedy:** Auf jeder Stufe des Baumes wird der optimale Split gemacht ohne vorzuschauen, welchen Effekt weitere Schritte haben würden.

Algorithmus

1. Berechne den Wert von **RSS** auf dem gesamten Trainingsdatensatz
2. Finde dasjenige Feature X_j und zugehörigen Schwellwert s , so dass die Aufteilung der Trainingsdaten in

$$x = (x_1, \dots, x_p) : x_j < s \quad \text{und} \quad x = (x_1, \dots, x_p) : x_j \geq s$$

den initialen RSS am meisten reduziert

3. Wiederhole das Verfahren für die in Schritt 2. erzeugten Regionen und zugehörigen Trainingsdaten
4. Dieser Prozess wird so lange fortgesetzt, bis ein vorher definiertes Abbruch-Kriterium (z.B. maximale Baumtiefe) erreicht worden ist
5. Berechne pro Blatt den durchschnittlichen Wert der Zielvariable für alle Trainingsdaten im Blatt

Anwendung

Leite den neuen Datenpunkt durch den Baum zum zugehörigen Blatt und weise den Prognosewert zu

Anpassung für Klassifikation

- Finale Entscheidung: Majority vote statt Mittelwert (wie bei k -NN)
- Loss Function: **Gini-Index** oder **Entropie**. Für beide ist der Wert klein, wenn die **Datenpunkte in einem Knoten überwiegend zu genau einer Klasse gehören**

Hyperparameter

Diskussion (2-3 Min.)

Welche Hyperparameter können Sie sich bei einem Entscheidungsbaum vorstellen?

► Hyperparameter Entscheidungsbaum

- ``max_depth``: maximale Tiefe des Baums
- ``min_samples_split``: minimale Anzahl an Samples, die benötigt werden, um einen Knoten zu splitten
- ``min_samples_leaf``: minimale Anzahl an Samples, die in einem Blatt vorhanden sein müssen
- ``max_features``: maximale Anzahl an Features, die für einen Split berücksichtigt werden
- ``criterion``: Kriterium zur Bewertung der Qualität eines Splits (z.B. (M)SE für Regressionsbäume)
- ...

Kategorische Variablen

- Entscheidungsbäume eignen sich auch für kategorische Variablen
- Leider ist für Anwendung in `sklearn` Umwandlung mittels `LabelEncoder` oder `OneHotEncoder` notwendig
- Achtung: Gruppierung von Datenpunkten teilweise notwendig, um sog. *long tail problem* (viele seltene Werte führen zu vielen zusätzlichen Features) zu vermeiden

Overfitting

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=12)
```

```
dt = DecisionTreeRegressor(max_depth=2)
dt.fit(X_train, y_train)
np.round(dt.score(X_train, y_train), 2)
```

0.6

```
np.round(dt.score(X_test, y_test), 2)
```

0.61

```
dt = DecisionTreeRegressor(max_depth=20)
dt.fit(X_train, y_train)
np.round(dt.score(X_train, y_train), 2)
```

1.0

Wenn wir maximale Baumtiefe von 20 erlauben, bekommen wir ein nahezu perfektes Modell auf den Trainingsdaten, mit nur 2 Prädiktor-Variablen! (Auch mit deutlich weniger als 20)

```
np.round(dt.score(X_test, y_test), 2)
```

0.53

Das Modell performt schlechter auf Testdaten, d.h. schlechte Generalisierbarkeit durch Over-Fitting.

Vor- und Nachteile von Entscheidungsbäumen

- (+) Einfach zu erklären; sogar einfacher als lineare Regressionsmodelle
- (+) Passen gut zur Entscheidungsfindung von Menschen
- (+) Können visualisiert und selbst von nicht-Expert*innen interpretiert werden (außer sie sind zu groß)
- (+) Bäume können (eigentlich) mit kategorischen Variablen umgehen
- (-) Entscheidungsbäume sind nicht sehr robust, d.h. kleine Änderungen in Daten können zu deutlich anderen Bäumen führen (Hohe Varianz, Overfitting)

Regularisierung durch Pruning

- Idee: Schneide den Entscheidungsbaum auf einen Teilbaum zurück (d.h. entferne manche Knoten), so dass dieser einen möglichst geringen Cross Validation Fehler auf der Testmenge liefert
- D.h. Auswertung auf einer Testmenge ist Teil der Regularisierung
- In `sklearn` implementiert über den Hyperparameter `ccp_alpha`

Regularisierung durch Ensemble-Methoden

- Es gibt verschiedene **Erweiterungen** von Entscheidungsbäumen, die die Varianz reduzieren und somit robuster sind
- Idee: statt eines einzelnen Baums trainieren wir ein **Ensemble von Bäumen** und kombinieren ihre Entscheidungen
- Übliche Verfahren: Bagging, Boosting, Random Forests

Bagging

- **Bootstrapping**: Erzeuge n Trainingsmengen T_i gleicher Größe durch zufälliges Ziehen mit Zurücklegen aus der ursprünglichen Trainingsmenge (n **bootstrapped training sets**)
- Trainiere n Bäume (ein Baum pro Trainingsmenge T_i) und kombiniere ihre Ergebnisse
- Zusätzlicher Hyperparameter: n : Anzahl an Bäumen (bzw. Bootstraps)

Out-Of-Bag (OOB) estimates

- man kann zeigen, dass jeder neue Trainingssatz T_i ca. $1/3$ der gesamten Trainingsdaten auslässt
- ein Datensatz, welcher in der Trainingsmenge für einen `bagged tree' nicht vorkommt, heißt **out-of-bag** (OOB)
- OOB ermöglicht die Berechnung eines **validen Test-Scores** ohne weitere Aufteilung der Daten: für jeden Datenpunkt x betrachte alle `bagged trees', für die x OOB ist. Berechnen die Vorhersage für x nur mithilfe dieser Bäume.

Vor-/Nachteile von Bagging

- (+) Generalisierbarkeit verbessert sich
- (-) Interpretierbarkeit verschlechtert sich, insbesondere wenn viele Bäume verwendet werden. Aber: es lässt sich

eine zusammenfassende Beschreibung der Relevanz der einzelnen Features generieren: **Feature Relevance / Variable Importance**

Random Forests



- Bäume werden ebenfalls mithilfe von bootstrapped Trainingsmengen trainiert
- Unterschied: bei jedem Split wird nur eine zufällig ausgewählte Teilmenge an Prädiktoren berücksichtigt

- Die Teilmenge von zu berücksichtigenden Prädiktoren wird für jeden Split neu erstellt
- Anzahl an Prädiktoren m pro Split typischerweise $m \sim \sqrt{p}$
- Zusätzlicher Hyperparameter: m : Anzahl an Prädiktoren pro Split

Der Großteil der Prädiktoren wird bei jedem Split ignoriert. Dies ist zwar kontra-intuitiv, sorgt aber für Regularisierung:

- Angenommen der Prädiktor x_j ist besonders wichtig und einige andere nur moderat wichtig.
- $\Rightarrow$ Beim Bagging würden die meisten Bäume beim obersten Split den Prädiktor x_j verwenden.
- $\Rightarrow$ Die Bäume würden sich ähneln, Vorhersagen der verschiedenen Bäume wären stark korreliert. Die Varianz bleibt relativ hoch.
- Beim RF würden ca. $\frac{p-m}{p}$ den starken Prädiktor x_j gar nicht verwenden, andere hätten dann eine Chance ins Gewicht zu fallen.

Boosting

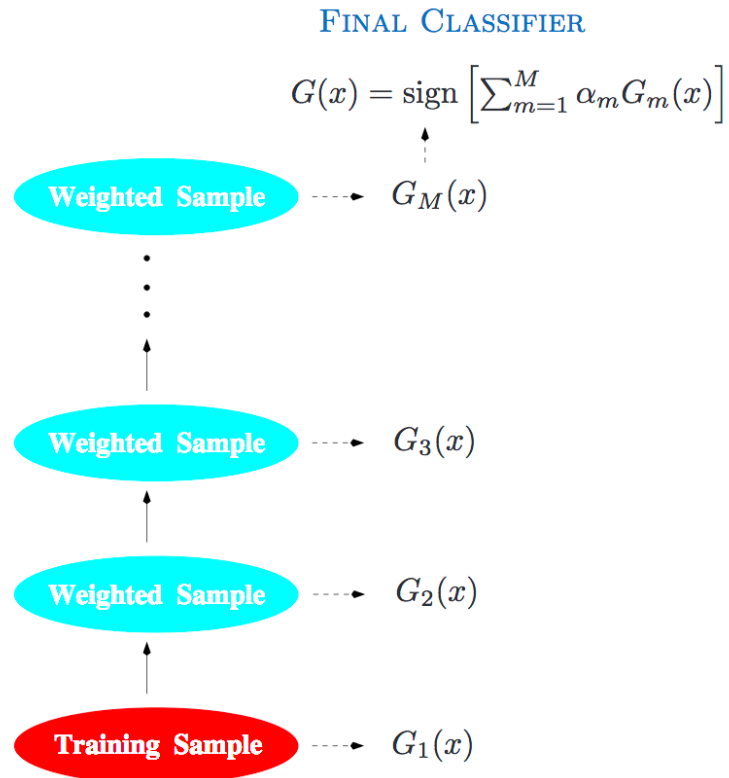
- Eines der effektivsten nicht-neuronalen Lern-Verfahren der letzten 20-30 Jahre
- Funktioniert ebenfalls für Klassifikation und Regression
- Idee: produziere ein Ensemble von vielen schwachen Modellen, welche gemeinsam ein "mächtiges Komitee" bilden
- Oberflächlich ähnelt es Boosting aber die Algorithmen funktionieren deutlich anders

AdaBoost.M1 (Freud und Schapire 1997)

- angenommen wir haben ein binäres Klassifikationsproblem mit Zielvariable $y \in \{-1, 1\}$
- ein schwacher Klassifikator performt nur wenig besser als Zufall
- Idee von Boosting ist, den schwachen Klassifikator auf modifizierte Varianten der Trainingsdaten wiederholt

anzuwenden

- Das ergibt eine Sequenz von schwachen Klassifikatoren, deren Entscheidungen gewichtet kombiniert werden:



Gewichtung

- Die Vorhersagen der Klassifikatoren G_i werden gewichtet; die finale Vorhersage entspricht der gewichteten Summe der einzelnen Vorhersagen
- Je genauer die Vorhersagen eines Klassifikators auf der Trainingsmenge, desto höher seine Gewichtung α_i
- In jedem Trainings-Schritt werden die Trainingsdaten gewichtet: im initialen Schritt ist jeder Datensatz gleich

gewichtet. Die Gewichte werden in jedem Schritt so angepasst, dass die im vorherigen Schritt falsch klassifizierten Datenpunkte höher gewichtet werden. Die Klassifikatoren haben also die Aufgabe, Datensätze, die die Vorgänger nicht richtig vorhersagen konnten, richtig zu klassifizieren. Die Klassifikatoren später in der Kette haben also die Aufgabe, die "schwierigeren" Datenpunkte korrekt zu klassifizieren.

Aufgaben

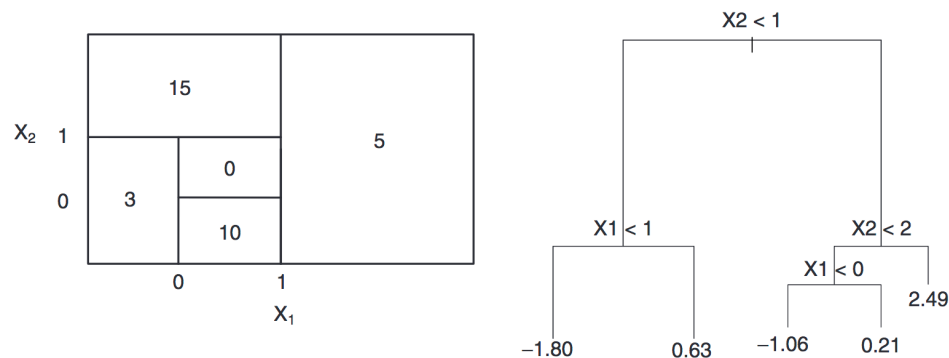
Aufgabe 1 - Partition des Feature-Raums

Konstruieren Sie ein Beispiel für einen Datensatz mit 2 Features und erstellen Sie eine Partition des Feature-Raums in 6 Rechtecke, welche durch die Anwendung eines Decision Tree Verfahrens (recursive binary split) entstanden sein konnte. Labeln Sie im Graphen die Rechtecke mit $R_1, \dots, R_6$ sowie die zugehörigen Schwellwerte.

Aufgabe 2 - Entscheidung im Ensemble

Sie haben ein Ensemble (Bagged Trees oder Random Forest) bestehend aus 10 Entscheidungsbäumen für Klassifikation eines Datensatzes mit 2 Klassen, grün und rot. Für einen Datenpunkt x liefern die 10 Bäume folgende Wahrscheinlichkeiten für die Zugehörigkeit zu Klasse rot: $(0.1, 0.2, 0.15, 0.55, 0.6, 0.2, 0.75, 0.7, 0.6, 0.5)$. Es gibt nun 2 Verfahren, um die Ergebnisse der einzelnen Bäume zu kombinieren: Mehrheitsvotum sowie Mittelwert der Wahrscheinlichkeiten und danach die Zuordnung zur Klasse mit Wahrscheinlichkeit > 0.5 . Zu welchem Ergebnis würden die beiden Verfahren jeweils führen?

Aufgabe 3 - Von Feature-Partition zu Entscheidungsbaum



1. Skizzieren Sie einen Entscheidungsbaum für Regression basierend auf der Partition des Feature-Raums im Bild links. Die Zahlen in den Rechtecken sind die Mittelwerte der Zielvariable.
2. Skizzieren Sie eine Partition des Feature-Raums zum Entscheidungsbaum im Bild rechts.

Aufgabe 4 - Implementierung

1. Trainieren Sie einen Entscheidungsbaum auf dem Palmer Penguins Datensatz, mit vergleichbarer Auswahl von Features und Zielvariable wie in bisherigen Hausaufgaben. Die Baumtiefe darf maximal 4 betragen. Visualisieren Sie den trainierten Baum. Bonus: Machen Sie sich mit der Python-Bibliothek graphviz vertraut und erstellen Sie eine Visualisierung des Baums mit dieser Bibliothek.
2. Berechnen Sie verschiedene geeignete Fehlermetriken auf der Trainings- und der Testmenge.